WOD2Sim: Adapting WOD-Style Driving Policies to Closed-Loop AlpaSim Evaluation

Alba Maria Tellez Fernandez
amtellezfernandez@gmail.com

Abstract: Closed-loop simulation exposes interface failures that open-loop driving-policy APIs hide. Waymo Open Dataset (WOD)-style policies consume assembled camera and ego context, route intent or geometry, and short-horizon ego trajectories, while AlpaSim external drivers are live services that receive incremental sensor and route messages and return trajectories at the rollout clock. *WOD2Sim* is an adapter contract and reference implementation for that boundary. It preserves route waypoints at prediction time, reconstructs policy-facing scene signals, resamples trajectories to simulator cadence, isolates optional model backends during discovery, hardens session lifecycle handling, and records launch provenance. The release ships dependency-light baselines, learned-policy adapter entry points, setup/readiness/launch commands, conformance tests, a synthetic public demo, and audit, support-bundle, and benchmark-readiness artifacts. WOD2Sim is not a new policy, scene converter, or public benchmark result; benchmark claims require executed multi-scene rollouts. Its contribution is an auditable path from WOD-style trajectory policies to AlpaSim external drivers, separating adapter-contract failures from policy performance.

1 Introduction

The Waymo Open Dataset (WOD) end-to-end driving task and AlpaSim solve different problems, so they expose different interfaces. WOD gives a policy-learning contract: observe camera and ego context, condition on route intent, and predict a short-horizon trajectory [1]. AlpaSim gives a simulator-execution contract: a live driver service is discovered as a trajectory-model plugin, receives camera and egomotion messages over time, processes route updates, returns trajectory points and headings at the rollout clock, and remains alive inside a distributed runtime [2].

Throughout this paper, *WOD-style* refers to the policy-interface shape: logged observations, route intent or route geometry, and short-horizon ego-relative trajectory outputs. It is not a claim of official Waymo challenge compatibility, leaderboard submission support, or complete Waymo scene reconstruction inside AlpaSim.

That difference is the reason a direct policy wrapper was insufficient. A dataset policy usually expects a compact, already-assembled example. A simulator driver must survive an ongoing session and consume state as it arrives. In stock AlpaSim, route geometry had already been collapsed into a left/straight/right command before prediction, which is acceptable for command-conditioned models but not for policies that score trajectories against a local route polyline. Even after route waypoints were exposed, another mismatch appeared: multi-lane road centers were not necessarily ego travel lanes. The remaining gaps were also between a dataset contract and a simulator contract: importing the model package could require unselected Alpamayo or VAM backends, late session messages could terminate batch runs, and launch assumptions lived outside the experiment record.

This paper is about extending that simulator-side contract so WOD-style policies can run as ordinary AlpaSim external drivers. WOD2Sim preserves route geometry at prediction time, decouples external plugin discovery from unrelated built-in backends, makes session handling idempotent under normal

WOD-AlpaSim manuscript 2026, Austin TX, USA.

Contract point	Stock AlpaSim behavior	WOD2Sim behavior
Route input	route used mainly for command	route waypoints preserved for prediction
Model imports	eager built-in backend imports	lazy imports for external plugin envs
Plugin boundary	built-in model stack	local `alpasim.models` policies
Session messages	unknown-session errors	late messages handled idempotently
Trajectory output	simulator frequency contract	WOD-style 5 s horizon resampled to runtime
Deployment	wizard-driven built-ins	matched external driver and wizard commands
Host setup	monolithic dependency assumptions	minimal editable install and overrides

Table 1: External-driver contract gaps exposed by WOD-style policies. The adapter keeps AlpaSim's driver loop but extends the state available at its boundary.

service ordering, and records launch state before rollout. On top of that contract, the project registers a learned token BC/DAgger policy and a direct actor-aware planner through `alpasim.models`, reconstructs a policy-facing signal from AlpaSim inputs, and provides setup, readiness, launch, and contract-check commands. The artifact claim is narrow: this is not a new autonomy method or policy benchmark, but a reusable external-driver contract extension for turning dataset-style trajectory policies into executable AlpaSim drivers.

The contributions are:

- an AlpaSim external-driver contract extension for WOD-style trajectory policies, including route geometry, trajectory resampling, and policy-facing scene signal;
- simulator-runtime hardening for lazy model discovery, late session messages, and idempotent close behavior that would otherwise corrupt batch evaluation;
- launch-state materialization and readiness checks for reproducing runs under local Docker, GPU, scene-artifact, and checkout constraints;
- an evidence pipeline that emits manifests, audits, support-bundle reports, hashes, and benchmark-summary artifacts while avoiding redistribution of gated assets.

2 External-Driver Contract Gaps

Stock AlpaSim already has the right outer loop: the runtime asks an ego-driver service for a trajectory, and the controller executes it. The failure was inside that loop. The driver input did not carry the route geometry our policies used, model discovery pulled in built-in backends before our plugin was selected, and the service treated normal late messages as fatal unknown-session errors. Launch state was also distributed across Hydra, Docker, GPU runtime, and scene-artifact assumptions. Table 1 lists the contract points we had to change.

2.1 Route Geometry Must Survive Prediction

Stock AlpaSim and WOD-style policies do not consume equivalent route representations. In the original driver API, `PredictionInput` contained camera images, command, speed, acceleration, and ego pose history. The route had already been reduced to a high-level driving command such as left, straight, or right. That abstraction is sufficient for command-conditioned models, but it discards the local waypoint geometry needed by policies that generate trajectory points relative to a route centerline.

This loss cannot be repaired downstream in policy code. Once the route has been reduced to a command, the polyline geometry that distinguishes two legal left turns, lane offsets, curvature, and local alignment is no longer available to the model. The required change is therefore an extension of

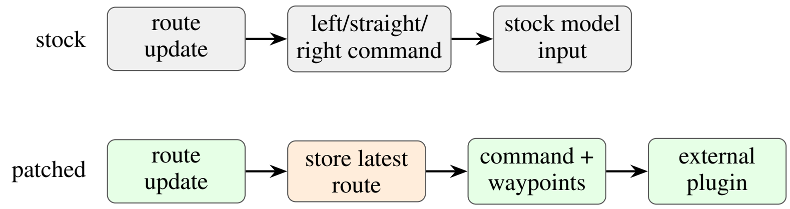

Figure 1: The key AlpaSim driver-contract patch preserves route waypoints for prediction instead of reducing route input to a command-only signal.

the simulator-facing prediction contract: route waypoints must remain available at prediction time. Figure 1 shows this contract change. The command remains available, but the route waypoints are preserved as first-class model input rather than reconstructed from a categorical intent symbol.

The implementation also exposed a second route error: a road centerline is not always the ego travel lane. In multi-lane scenes, scoring trajectories against the geometric road center can make a legal ego lane appear laterally offset or unsafe. The adapter therefore treats route waypoints as the policy route and derives route-centered lane geometry for policy scoring, instead of assuming that the simulator's visual road center is the trajectory reference.

2.2 Model Discovery Must Not Import Unselected Backends

The second incompatibility is in model discovery. Stock AlpaSim's model package assumes that built-in learned-driver backends are available when the package is imported. That is reasonable for the simulator's native stack, but it breaks the external-driver case. A local WOD-style plugin may need only `BaseTrajectoryModel`, `PredictionInput`, and the `alpasim.models` entry point. If those imports also load Alpamayo or VAM, the driver can fail before the requested external model is selected.

WOD2Sim therefore separates the stable base contract from heavyweight backend availability. External driver execution requires `BaseTrajectoryModel`, `PredictionInput`, `ModelPrediction`, and related base types to be importable without installing every built-in model backend. This makes the plugin boundary explicit: external policies can register through the same AlpaSim mechanism as native models while depending only on the selected driver contract. Learned WOD2Sim plugins may still require torch; the removed coupling is to unrelated built-in policy backends that are not needed by a selected external plugin.

2.3 Session Lifecycle Must Be Idempotent

The third incompatibility appears only once the code runs as a simulator service. A script-style converter can assume one ordered control flow. AlpaSim rollouts involve a runtime, driver service, controller, renderer, and deployment orchestration exchanging session-scoped messages asynchronously. Duplicate closes and late image, egomotion, or route messages are normal service-lifecycle events, not necessarily policy failures.

Treating those messages as fatal unknown-session errors makes closed-loop batch evaluation fragile for reasons unrelated to policy quality. The driver service must therefore make session handling idempotent at the runtime boundary: late messages should be recorded as controlled warnings and ignored when their session is no longer active. Without this session behavior, repeated external-driver runs fail for service-lifecycle reasons rather than for policy errors.

2.4 Launch State Must Be Materialized

The final contract gap is outside the model API. A local AlpaSim external-driver run depends on a matched driver command, wizard command, topology, base port, scene IDs, Docker image, GPU runtime, platform constraints, and gated scene artifacts. If those values live only in shell history or ad

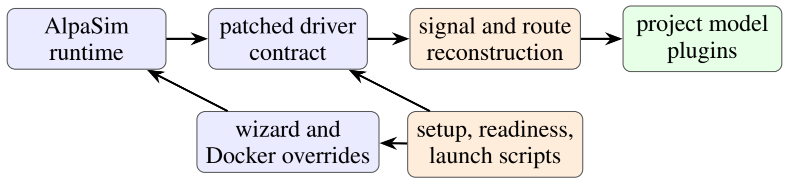

Figure 2: Implemented external-driver contract. Patched AlpaSim code preserves route and runtime state; project plugins consume a compact policy-facing signal.

hoc Hydra overrides, the policy cannot be re-executed as a reviewable artifact. WOD2Sim therefore treats launch state as part of the external-driver contract: setup checks plugin discovery, readiness checks runtime preconditions, and print mode emits the exact driver and wizard commands before rollout.

3 Implemented Contract

Figure 2 shows the implemented contract boundary. The implementation has two halves: changes inside the AlpaSim checkout, and project code that behaves like ordinary AlpaSim model plugins.

3.1 AlpaSim-Side Changes

The patched AlpaSim checkout contains four relevant changes.

Route waypoint propagation. The driver session now stores the most recent route and exposes it through `route_waypoints_for_prediction`. The batch prediction path copies those waypoints into `PredictionInput`. This preserves the route geometry generated by AlpaSim's runtime and makes it available to every external model plugin.

Dependency-light model registry. The model package keeps `BaseTrajectoryModel`, `PredictionInput`, `ModelPrediction`, `DriveCommand`, and `ManualModel` importable without importing all heavyweight backends. Alpamayo and VAM remain available through lazy access when installed. External plugins therefore need only the base driver abstractions plus the shared driver dependencies installed by the setup script.

Asynchronous session hardening. The driver now handles unknown-session close, image, egomotion, and route messages as late or duplicate messages rather than immediate crashes. During batch runs, the simulator, driver, controller, and renderer do not always stop at the same microsecond; the driver records those messages as controlled lifecycle events.

Local external-driver deployment. The override layer adds deployment and Docker changes for local external-driver use, including host/wizard command generation, ARM-aware deploy selection, minimal Docker dependency groups, and GPU reservation behavior that matches local topologies. These changes move launch state out of ad hoc shell commands and into files that can be printed, inspected, and rerun.

3.2 Project-Side Plugins

The project registers four AlpaSim model entry points: `constant_velocity`, `route_following`, `token_dagger_bc`, and `direct_actor_planner`. Each implements AlpaSim's `BaseTrajectoryModel.from_config` factory and returns `ModelPrediction` objects with trajectory points, headings, and structured reasoning metadata.

Dependency-light baselines. `constant_velocity` and `route_following` require no checkpoint, proxy, torch import, or private experiment state. They exist to make smoke execution and benchmark-readiness baselines auditable through the same route, sensor-freshness, trajectory-resampling, launch, and JSONL evidence paths as learned policies.

Token BC/DAgger plugin. The learned plugin loads token-policy checkpoints, normalizes geometric features, combines learned logits with route and scene-signal checks, and records per-run selection logs. Public runs supply the checkpoint path explicitly; the release does not ship checkpoints or named experiment variants.

Direct actor-planner plugin. The selector-free planner bypasses token logits and samples a small continuous trajectory family. It scores candidates with actor, route, lane, smoothness, progress, speed, and rear-flow terms, then returns the best trajectory under the same AlpaSim driver contract. This lets tokenized and direct planning interfaces run inside the same simulator instead of separate evaluation scripts.

Across these plugins, route scoring uses policy-facing route centerlines rather than raw road-center geometry. Exposing a route polyline is not enough; the policy must score against the lane the ego should actually follow.

The code split follows the contract split. Route propagation, model discovery, session lifecycle, and deploy behavior are patched in the AlpaSim checkout through `route_waypoints.patch` and `local_checkout.patch`. Policy registration, signal reconstruction, and policy logic stay in project code through `pyproject.toml`, `alpasim_signal.py`, `alpasim_contract.py`, `maneuver_candidates.py`, `alpasim_token_bc.py`, and the direct planner adapter. Setup, readiness, and launch commands then make that combined surface runnable as one external-driver artifact.

4 Policy Signal

Dataset policies do not consume raw AlpaSim driver messages directly, so the adapter builds a compact policy signal from `PredictionInput`. It extracts route waypoints from the patched AlpaSim field when present. If route geometry is unavailable, the adapter can still build a command-proxy route so the driver remains debuggable, but the frame is tagged with `route_source=command_proxy` and the public audit gate rejects it as claim-valid evidence. The signal also records speed, acceleration, pose-history length, camera count, diagnostic visibility risk from camera brightness, and optional structured hazards from fields such as `structured_hazards`, `hazards`, `traffic_hazards`, or `alpasignal`.

Structured hazards are converted into the internal policy representation. Static hazards become obstacles with shape and heading metadata. Moving hazards become actors with velocity, size, heading, and inferred or explicit behavior. Route waypoints become the local lane centerline used by the policy and planner. Visibility and dynamics risk are diagnostic signals only; without an explicit structured hazard, the adapter does not fabricate obstacles. This translation is intentionally independent of WOD protobufs and AlpaSim internals: it is the shared policy input used by both AlpaSim plugins.

5 Trajectory Output and Resampling

The output side of the adapter is intentionally small. WOD-style policies return ego-relative trajectory samples over a fixed five-second horizon, while AlpaSim expects trajectory points and headings on the simulator runtime grid. WOD2Sim interprets policy points as endpoint samples at $(i/N)H$, where H is the horizon and N is the policy point count. The target grid contains $\mathrm{round}(f_{\mathrm{runtime}}H)$ endpoint samples. If the source and target counts match, the adapter returns the policy trajectory unchanged. Otherwise, it anchors interpolation at the current ego origin $(0, 0)$, linearly interpolates x/y positions onto the target endpoint grid, and recomputes headings from adjacent runtime points using AlpaSim's trajectory-model helper.

This contract is tested rather than assumed. The suite checks exact identity for native runtime sample counts, exact interpolation for linear endpoint trajectories within `float32` tolerance, and a replay-identity adapter that returns logged trajectory points through the same `ModelPrediction` path without distorting positions. For curved trajectories, the positional error is the ordinary piecewise-linear interpolation error induced by the source sample spacing and path curvature; this release does not claim a closed-loop rollout accuracy bound from resampling alone.

6 Launch Contract

The setup command validates that `ALPASIM_ROOT` is a real checkout, applies the repo-tracked AlpaSim patches and overrides, bootstraps the AlpaSim virtual environment, compiles protobuf modules, installs the required AlpaSim packages editably, installs this repository as a no-dependency editable package, and verifies that the required `alpasim.models` entry points are discoverable.

The readiness command checks the AlpaSim checkout shape, Docker access, platform compatibility, the local AlpaSim base image, NVIDIA container runtime visibility, and scene artifacts. The run launcher then materializes an `external-driver-config.yaml`, `driver-command.sh`, `wizard-command.sh`, and `launch-metadata.json`. The metadata records the AlpaSim checkout git state and local AlpaSim base-image inspection result when available, so command plans carry the simulator and Docker provenance needed for later audit. It supports named scene presets plus the public adapters `constant_velocity`, `route_following`, `token_dagger_bc`, and `direct_actor_planner`. The driver and wizard are launched as a matched pair: the driver runs from the AlpaSim virtual environment, while the wizard receives the external driver address, topology, scene IDs, log directory, base port, and timeout.

These files are part of the artifact. Without them, an AlpaSim run depends on a hidden mixture of Hydra overrides, Docker image state, plugin installation state, gated scene artifacts, and host-specific GPU behavior. WOD2Sim writes those assumptions before the rollout starts.

7 Contract Validation

The tests check the extended contract rather than driving performance. Table 2 lists the checks. Each row corresponds to a contract property that must hold before policy comparisons inside AlpaSim are meaningful.

The validation suite is split into two tiers. Core adapter checks cover route propagation, route-lane semantics, signal reconstruction, plugin registration, setup planning, launch materialization, and non-learned adapter outputs; these tests collect and run in a dependency-light environment. The repository exposes this tier as `make conformance`, which intentionally excludes torch, checkpoints, Docker, GPUs, and gated scenes. Learned-policy executable checks cover token BC/DAgger checkpoints, tensor inference, token-schema validation, and selection logs; those tests are torch-gated and skip cleanly when the deep-learning runtime is not installed.

The repository also includes `make demo`, an ungated synthetic contract demo that generates a driver log, route audit, aggregate JSON, support bundle, and SVG rollout view without launching AlpaSim or loading a checkpoint. Its aggregate JSON also reports deterministic geometry diagnostics for command-proxy route loss and road-center versus ego-route offset on public synthetic waypoints. The demo is useful for inspecting the artifact layout and contract failure modes, but its metadata keeps `valid_claim_evidence=false` and it is not a policy-quality result.

The minimal audit path has four stages. First, run the setup command in check-only mode to verify that the AlpaSim checkout can see the local model entry points. Second, run the readiness command to check Docker, GPU runtime, scene artifacts, and platform assumptions. Third, launch in print mode to materialize the exact driver and wizard commands without starting a rollout. Finally, run the AlpaSim integration and setup-script tests to verify the core checks above. In environments with torch installed, the same test module also executes the learned token-policy checks; without torch, those

Contract property	Check
Local policies are discoverable by AlpaSim	Integration tests check the `alpasim.models` registry; setup tests verify discovery in the AlpaSim virtual environment.
Route geometry reaches policy code	Signal tests construct prediction inputs with route waypoints and check `alpasim_waypoints` as the route source.
Route scoring follows the ego travel lane	Scenario tests check that multi-lane routes use policy-facing `route_centerline` geometry instead of raw road-center interpolation.
Scene signal is policy-facing, not protobuf-facing	Tests pass structured hazards through multiple input aliases and verify static obstacles, moving actors, shape metadata, headings, velocities, and inferred behaviors.
Core adapters satisfy the trajectory-model contract	The direct planner adapter, resampling identity checks, endpoint-interpolation checks, and replay-identity adapter return trajectory arrays, headings, and structured reasoning payloads through the AlpaSim model API without learned-checkpoint runtime requirements.
Learned-policy executables are contract-checked	Torch-gated token-policy tests verify checkpoint loading, token names, feature normalizers, device selection, tensor inference, and selection-log outputs; bad token schemas are rejected.
Launch state is materialized and checkable	Setup-script tests cover checkout validation, override application, protobuf compilation, editable install planning, driver command generation, wizard command generation, scene preset loading, Docker/image preflights, and platform guards.

Table 2: Contract validation. Core checks run without the learned-policy runtime; token BC/DAgger checks are gated on torch.

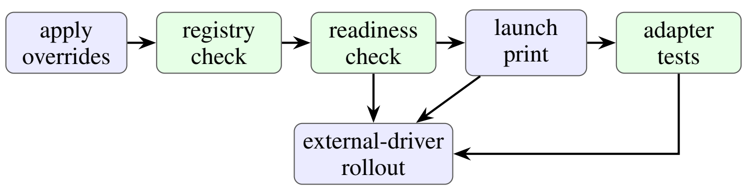

Figure 3: Audit path. Setup, readiness, command materialization, and tests isolate integration failures before closed-loop rollouts.

tests are skipped rather than blocking collection. A failed core check points to a specific boundary: patch application, plugin discovery, runtime prerequisites, launch materialization, route propagation, or adapter semantics.

Figure 3 summarizes this audit path. A single successful rollout does not validate the contract. The staged checks show whether a failure came from patch application, plugin discovery, runtime prerequisites, launch materialization, or policy signal conversion.

8 Related Work

WOD2Sim sits between existing closed-loop simulators, benchmark agent APIs, and dataset-first policy interfaces. nuPlan defines a planner abstraction and closed-loop benchmark around map and scenario state [3]. CARLA and its leaderboard ecosystem define an agent interface for an interactive simulator with a different sensor and route stack [4]. NAVSIM evaluates vision-based driving policies in a non-reactive simulation protocol that scales better than full closed-loop execution but does not expose the same persistent driver-service lifecycle [5]. Waymax provides accelerated data-driven simulation over WOD-style scenarios and emphasizes large-scale in-graph training and evaluation interfaces [6].

Category	Report	Purpose
Baselines	open-loop WOD-style evaluation, replay or constant-velocity driver, route-following driver, stock AlpaSim external-driver path, and WOD2Sim ablations without route reconstruction or lifecycle hardening	show what closed-loop execution and the adapter contract add beyond a simple API wrapper
Closed-loop metrics	collision rate, off-road rate, route progress, completion, timeout rate, valid-frame ratio, sensor freshness, action latency, late-message rate, and support-bundle validity	separate policy behavior from runtime and evidence failures
Scene coverage	straight, turn, lane-change, dense-traffic, occlusion, and route-merge scenes across at least a small multi-scene set	support claims beyond a single recorded scene
Failure taxonomy	route drift, stale observations, heading-error compounding, recovery failure, lifecycle crashes, and audit invalidation	expose failures hidden by open-loop log replay

Table 3: Evaluation protocol for claims built on WOD2Sim. The current release ships the contract checks and evidence schema, but no policy benchmark result; benchmark claims require executed scenes, baselines, and complete failure reporting.

The WOD2Sim claim is narrower than those systems. It does not introduce a simulator, planner benchmark, or new learned policy. Its contract contribution is preserving route geometry at the AlpaSim prediction boundary, keeping policy registration compatible with `alpasim.models`, handling external-driver lifecycle events, and materializing local launch provenance for audits. Those concerns are the parts that were missing when WOD-style trajectory policies were run as AlpaSim external drivers.

9 Evaluation Protocol and Claim Boundary

WOD2Sim should be evaluated as a closed-loop evaluation interface, not as a new driving model. The current public release does not publish executed-scene metrics. Its artifact claim is contract readiness: a WOD-style policy can be exposed as an AlpaSim external driver, receive route and scene state through a stable contract, return runtime trajectories, and produce auditable evidence when run with user-provided assets. A benchmark claim requires executed scenes, baselines, and closed-loop failure analysis. The release therefore includes `wod2sim-benchmark-readiness`, which fails until public-safe batch summaries satisfy a minimum executed-scene and baseline matrix. Table 3 summarizes the protocol that future benchmark claims must report; it is not a result table.

The claim boundary is therefore precise: WOD2Sim bridges WOD-style policy interfaces to AlpaSim closed-loop execution. It does not claim full Waymo scene reconstruction, a new simulator, or a new autonomy model. That boundary is important because a full Waymo-to-AlpaSim converter would need to reconstruct maps, actors, dynamic agents, traffic lights, sensor setup, and dataset terms at a scope beyond this artifact.

10 What Generalizes

WOD2Sim is specific to this project, but the adapter pattern is not. The same external-driver shape should transfer to other dataset-trained or benchmark-trained trajectory policies that are not written as native AlpaSim drivers.

The reusable part is the boundary definition: preserve simulator route geometry at prediction time; keep policy code independent of dataset protobufs and simulator message internals; register policies through the existing `alpasim.models` mechanism; separate base driver abstractions from optional heavyweight backends; handle late session messages idempotently; and materialize launch state before rollout. Those choices let very different policies share one closed-loop runtime. In this artifact, the learned token policy and direct actor planner run through the same route signal, resampling path, and launch system.

The result is therefore more than a WOD integration. It is a concrete reference for what an AlpaSim external-driver adapter has to provide when the source policy was designed for offline learning or log-based evaluation rather than for a persistent simulator service.

11 Limitations

WOD2Sim does not claim to reconstruct the full WOD world state inside AlpaSim. Its primary claim is execution compatibility: making WOD-style and project policies run as native AlpaSim drivers. Richer traffic reconstruction, broader WOD subset release, dataset-terms packaging, and standard public baselines are natural next steps. The current contract exposes the important systems boundary: route geometry, trajectory timing, plugin loading, deployment, and simulator-driver interaction.

12 Conclusion

WOD2Sim extends AlpaSim's external-driver contract for WOD-style trajectory policies. The extension preserves route geometry at prediction time, decouples external plugin discovery from unselected built-in backends, treats late session messages idempotently, and records launch state before rollout. Project policies then run as ordinary `alpasim.models` plugins over a compact policy-facing signal. The result is a contract-checked adapter harness for learned token and actor-aware planning policies, and a reference for building additional AlpaSim external-driver adapters.

References

[1] P. Sun, H. Kretzschmar, X. Dotiwalla, A. Chouard, V. Patnaik, P. Tsui, J. Guo, Y. Zhou, Y. Chai, B. Caine, et al. Scalability in perception for autonomous driving: Waymo open dataset. In *IEEE Conference on Computer Vision and Pattern Recognition*, pages 2446–2454, 2020.

[2] NVIDIA Research. AlpaSim: Autonomous driving simulation for policy evaluation, 2026. Software release v2026.4.

[3] H. Caesar, J. Kabzan, K. S. Tan, W. K. Fong, E. Wolff, A. Lang, L. Fletcher, O. Beijbom, and S. Omari. nuPlan: A closed-loop ml-based planning benchmark for autonomous vehicles. In *CVPR Workshop on Autonomous Driving*, 2021.

[4] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun. CARLA: An open urban driving simulator. In *Conference on Robot Learning*, pages 1–16, 2017.

[5] D. Dauner, M. Hallgarten, T. Li, X. Weng, Z. Huang, Z. Yang, H. Li, I. Gilitschenski, B. Ivanovic, M. Pavone, A. Geiger, and K. Chitta. NAVSIM: Data-driven non-reactive autonomous vehicle simulation and benchmarking. In *Advances in Neural Information Processing Systems*, 2024.

[6] C. Gulino, J. Fu, W. Luo, G. Tucker, E. Bronstein, Y. Lu, J. Harb, X. Pan, Y. Wang, X. Chen, J. D. Co-Reyes, R. Agarwal, R. Roelofs, Y. Lu, N. Montali, P. Mougin, Z. Yang, B. White, A. Faust, R. McAllister, D. Anguelov, and B. Sapp. Waymax: An accelerated, data-driven simulator for large-scale autonomous driving research. In *Advances in Neural Information Processing Systems*, 2023.